

Lab Practice Exercises 2

Lectures 3 and 4 · Practice only — nothing here is collected or graded

Fall Semester 2026 · Dr. Narasimha Rao Vajjhala · nrao@aubg.edu · Office MB 253

BEFORE YOU START

Environment	JDK 25 (the current Long-Term Support release) and IntelliJ IDEA — the same setup as Lab 1. Check it with <code>javac -version</code> before Question 1.
What this covers	Lecture 3 (conditionals, logical operators, switch) and Lecture 4 (while, do-while, for, nested loops, break and continue). Questions 1 to 20 mix the two; Part B is organised by topic.
Feeding the samples in	Every sample here is a whole console session. Rather than retyping the input each time you fix something, put it in a text file and redirect: <code>java Program.java < input.txt</code> . The redirection is a feature of your shell — the program is not changed by it, and Scanner cannot tell the difference.
Style	Brace every <code>if</code> and every loop body, even the one-statement ones. Questions 21, 22 and 27 exist to show you why, and the habit is part of the mark on the assignments and the project.
Nothing to hand in	These exercises are for practice. There is nothing to submit, no deadline and no mark — keep your <code>.java</code> files somewhere you can find them again, and bring anything that will not work to the lab or to office hours.

Questions 1 to 20 are the core set and range across both of this week's sessions — tax brackets and grade ladders from Tuesday, sentinel loops and nested patterns from Thursday. Questions 21 to 30 cover the parts of Lectures 3 and 4 that the core set does not reach: the conditionals that compile and still do the wrong thing, switch expressions, short-circuiting, precedence, the three loop mechanics that catch everyone once, and what goes wrong when a loop counter is a double.

None of this is collected, so work at whatever pace suits you and skip anything you already find easy. Two suggestions all the same. Predict the output before you run anything in Part B — several of those questions are worth nothing at all if you run them first and reason afterwards. And do not skip the ones that ask you to break something on purpose: 21, 22, 27, 29 and 30 are the ones that pay off later, because you will meet all five of those faults again when they are not deliberate. The material is examinable even though the exercises are not.

Three questions reach a little ahead of where we are. Questions 12 and 15 walk through a String character by character, and Question 17 wants a two-dimensional array; all three are Week 4 topics. They are marked where they appear. Everything else needs only Lectures 3 and 4.

Part A · Core Exercises

Question 1 · Progressive Income Tax Calculator

Write a Java program that calculates the federal income tax for an individual based on their filing status and annual taxable income. The program must read an integer representing the filing status, where 0 corresponds to a single filer, 1 to married filing jointly, 2 to married filing separately, and 3 to head of household. The program must then read a double value representing the annual taxable income.

Using conditional statements, compute the tax progressively according to the appropriate tax brackets for the given filing status. If the user enters an invalid filing status, the program must report the problem and exit gracefully rather than crashing.

Use the 2026 brackets from the Lecture 3 slides. A filing status only decides where the six boundaries sit — the progressive arithmetic above them is identical for all four, so write it once.

EXPECTED OUTPUT (EXAMPLE)

```
Enter filing status (0-3): 0
Enter taxable income: 65000
Tax: 9012.00
```

Question 2 · Leap Year Range Analyzer

Develop a Java program that reads two integers representing a starting year and an ending year. The program must analyze each year in this range, inclusive, and determine whether it is a leap year based on standard leap year rules.

All leap years must be printed to the console with exactly ten values per line. After printing the leap years, the program must print the total count of leap years found.

Test it on the range 1896 to 2104 as well. If 1900 and 2100 appear in your output, your leap-year test is the naive `year % 4 == 0` and the century rule is missing.

EXPECTED OUTPUT (EXAMPLE)

```
Enter start year and end year: 1996 2024
1996 2000 2004 2008 2012 2016 2020 2024
Count: 8
```

Question 3 · Sentinel-Controlled Statistics Calculator

Write a program that continuously reads integer values from the user until the sentinel value 0 is entered. The sentinel value must not be included in any calculations. Using loop logic, the program must compute the number of values entered, their sum, the average formatted to two decimal places, the smallest value, the largest value, and the count of negative values.

If the sentinel value is entered as the first input, the program must handle this case without producing runtime errors.

EXPECTED OUTPUT (EXAMPLE)

```
Enter integers, 0 to finish: 5 -2 10 -7 0
Count: 4
Sum: 6
Average: 1.50
Min: -7
Max: 10
Negatives: 2
```

Do not seed `min` with 0 out of habit. Work out what that does when every value entered is negative, and test that case. Seeding from the first real value works, and so does `Integer.MAX_VALUE` — but be able to say why.

Question 4 · Guessing Game with Attempt Limit

Create a guessing game program in which the user attempts to guess a secret number. For deterministic testing, the program must first read the secret number, followed by a sequence of guesses. The user is allowed a maximum of seven attempts.

After each guess, the program must indicate whether the guess is too low, too high, or correct. If the user guesses the number correctly within the allowed attempts, display the number of attempts used. If the user fails, display an appropriate failure message.

EXPECTED OUTPUT (EXAMPLE)

```
Enter the secret number: 42
10 50 41 42
Too low
Too high
Too low
Correct
Attempts: 4
```

Once this works, replace the first read with `(int) (Math.random() * 100) + 1` and play it properly. Keep the version that reads the secret as well — you cannot test a program whose answer changes every run.

Question 5 · Two-Digit Lottery Evaluation

Write a program that simulates a two-digit lottery game. The program must read a two-digit lottery number and a two-digit guess. Using conditional logic, determine whether the guess matches the lottery exactly, matches both digits in any order, matches exactly one digit, or does not match at all.

The program must print the monetary award corresponding to the outcome: 10,000 for an exact match, 3,000 for both digits in any order, 1,000 for one digit, and 0 otherwise.

EXPECTED OUTPUT (EXAMPLE)

```
Enter the lottery number (two digits): 47
Enter your guess (two digits): 74
Award: 3000
```

The order of the branches is the whole question. Test 44 against 44 and make sure it pays 10,000 rather than falling into the any-order branch.

Question 6 · Chinese Zodiac Calculator

Write a Java program that reads an integer representing a year and determines the corresponding Chinese Zodiac animal. The calculation must be based on the remainder of the year divided by 12. Use a switch statement to map remainders to zodiac animals.

EXPECTED OUTPUT (EXAMPLE)

```
Enter a year: 2024
Dragon
```

Question 24 asks you to write this a second time as a switch expression. Keep your answer.

Question 7 · Hollow Rectangle Pattern Generator

Develop a program that reads the number of rows, number of columns, a border character, and a fill character. Using nested loops, print a hollow rectangle where the border is drawn using the border character and the interior is filled with the fill character.

EXPECTED OUTPUT (EXAMPLE)

```
Enter rows, columns, border char, fill char: 4 7 # .
#####
#.....#
#.....#
#.....#
#####
```

One boolean expression decides every cell. Work out what makes a position an edge before you write the loops, and the body becomes a single line.

Question 8 · Multiplication Table Generator

Write a program that reads an integer N and prints a formatted multiplication table from 1 to N. The table must include row and column headers and must be properly aligned using spaces. Nested loops must be used to generate the table.

Align the columns with `printf` field widths, not by counting spaces. `"-".repeat(...)` will draw the rule under the header without a third loop — work out how wide it has to be from the field width you chose.

EXPECTED OUTPUT (EXAMPLE)

```
Enter N: 5
  | 1  2  3  4  5
--+-+-----
1 | 1  2  3  4  5
2 | 2  4  6  8 10
3 | 3  6  9 12 15
4 | 4  8 12 16 20
5 | 5 10 15 20 25
```

Question 9 · Decimal to Hexadecimal Converter

Create a program that reads a non-negative decimal integer and converts it to its hexadecimal representation without using built-in conversion methods. The conversion must be performed using repeated division by 16 and loop logic.

Check your answer against `Integer.toHexString` afterwards — but the loop has to produce it first.

EXPECTED OUTPUT (EXAMPLE)

```
Enter a non-negative integer: 43981
ABCD
```

Test 0 as well. The natural loop shape prints nothing at all for 0, which is a defect worth finding yourself.

Question 10 · Greatest Common Divisor and Least Common Multiple

Write a program that reads two positive integers and computes their greatest common divisor using loop-based logic. After computing the GCD, compute and display the least common multiple.

EXPECTED OUTPUT (EXAMPLE)

```
Enter two positive integers: 16 24
GCD: 8
LCM: 48
```

Write the LCM as `a / gcd * b`, not `a * b / gcd`. Try both with 50000 and 48000: the first gives 1200000, the second gives -947483, because `a * b` has already overflowed `int` before the division happens.

Question 11 · Prime Number Sequence Generator

Write a program that reads an integer K and prints the first K prime numbers. The program must test each number for primality using loops and must print exactly ten prime numbers per line.

EXPECTED OUTPUT (EXAMPLE)

```
Enter K: 25
2 3 5 7 11 13 17 19 23 29
31 37 41 43 47 53 59 61 67 71
73 79 83 89 97
```

Stop trial division at the square root: `for (int d = 2; d * d <= candidate; d++)`. Ask yourself why no factor can hide above it before you accept the line.

Question 12 · Palindrome String Analyzer

Write a program that reads a full line of text and determines whether it is a palindrome. The program must ignore spaces, punctuation, and letter case. Loop logic must be used to compare characters from both ends of the string.

EXPECTED OUTPUT (EXAMPLE)

```
Enter a line of text: A man, a plan, a canal: Panama
Palindrome
```

Reaches ahead: this needs `charAt` and `length()`, which are Week 4 material — Liang chapter 4, on 1 October. Everything else about it is Week 2 loop logic.

Question 13 · Score Processor with Loop Control

Develop a program that reads integer scores until the sentinel value -1 is entered. Scores outside the range 0 to 100 must be counted as invalid and ignored. If three perfect scores are entered, the program must terminate early using `break`.

EXPECTED OUTPUT (EXAMPLE)

```
Enter scores, -1 to finish: 50 101 100 99 -5 100 100 80 -1
Valid: 5
Invalid: 2
Average: 89.80
Stopped: 3 perfect scores
```

Read the expected output against the input carefully. The 80 is never read at all, because the third perfect score ends the loop before the program reaches it — and that is the point of the question.

Question 14 · Body Mass Index Classifier

Write a program that reads a person's weight in kilograms and height in meters and computes the Body Mass Index. Based on the computed BMI, the program must classify the result using standard BMI categories.

EXPECTED OUTPUT (EXAMPLE)

```
Enter weight (kg) and height (m): 72 1.75
BMI: 23.51
Category: Normal
```

Write the ladder so each branch after the first tests only its upper bound. If your second branch reads `bmi >= 18.5` && `bmi < 25`, you have restated a bound the first branch already ruled out, and the ladder is doing no work for you.

Question 15 · Character Classification Counter

Create a program that reads lines of text until the line "END" is entered. The program must count and display the total number of vowels, consonants, digits, and spaces across all input lines.

EXPECTED OUTPUT (EXAMPLE)

```
Hello World 2026
Java!
END
Vowels: 5
Consonants: 9
Digits: 4
Spaces: 2
```

Reaches ahead: like Question 12, this walks a String character by character. Decide deliberately what you do with the exclamation mark — it is neither vowel, consonant, digit nor space.

Question 16 · Centered Number Pyramid

Write a program that reads an integer N and prints a centered numeric pyramid. Each row must contain numbers increasing from 1 up to the row number and then decreasing back to 1. Proper spacing must be applied to center the pyramid.

EXPECTED OUTPUT (EXAMPLE)

```
Enter N: 4
  1
 1 2 1
1 2 3 2 1
1 2 3 4 3 2 1
```

Three loops inside the row loop: the indent, the climb, the descent. Getting the indent right is the part that takes the thinking.

Question 17 · Matrix Row and Column Sum Calculator

Develop a program that reads the dimensions of a matrix followed by the matrix values. If any value is outside the range -1000 to 1000, the program must print an error message and terminate. Otherwise, compute and display the sum of each row and each column.

EXPECTED OUTPUT (EXAMPLE)

```
Enter rows and columns: 2 3
1 2 3
4 5 6
Row sums: 6 15
Col sums: 5 7 9
```

Reaches ahead: column sums need the values stored, so this one genuinely wants a two-dimensional array — Week 4 material, on 29 September. Attempt it if you are ahead; otherwise come back to it then.

Question 18 · Grade Distribution Histogram

Write a program that reads an integer representing the number of students, followed by their numeric scores. Using conditional logic, classify each score into a letter grade and print a histogram showing the frequency of each grade.

Use the usual scheme: A from 90, B from 80, C from 70, D from 60, F below that.

EXPECTED OUTPUT (EXAMPLE)

```
Enter the number of students: 8
95 82 67 70 58 88 90 100
A: 3 ***
B: 2 **
C: 1 *
D: 1 *
F: 1 *
```

Five named counters are enough — you do not need an array for this. `"*".repeat(count)` draws each bar.

Question 19 · Floating-Point Safe Series Summation

Create a program that reads a step value and an ending value and computes the sum of the series using an integer-controlled loop to avoid floating-point equality errors. The final sum must be printed to four decimal places.

That is, sum step, $2 \times \text{step}$, $3 \times \text{step}$ and so on up to the ending value.

EXPECTED OUTPUT (EXAMPLE)

```
Enter the step and the ending value: 0.1 1.0
Sum: 5.5000
```

Write it the wrong way first — `for (double d = step; d <= end; d += step)` — and print how many times the body ran. Question 30 is about exactly this.

Question 20 · Menu-Driven Utility Program

Develop a menu-driven program that repeatedly displays a menu allowing the user to select from several utilities, including computing the GCD of two numbers, converting a decimal number to hexadecimal, checking whether a number is prime, and printing a multiplication table. The program must continue executing until the user chooses to exit.

You have already written all four utilities. This question is about the loop around them.

EXPECTED OUTPUT (ABRIDGED)

```
1 GCD of two numbers
2 Decimal to hexadecimal
3 Is a number prime?
4 Multiplication table
0 Exit
Choice: 3
97
Prime

[menu redisplayed]
Choice: 4
4
  | 1 2 3 4
--+-----
1 | 1 2 3 4
2 | 2 4 6 8
3 | 3 6 9 12
4 | 4 8 12 16

[menu redisplayed]
Choice: 0
Bye
```

The menu has to be shown before the choice can be tested, so this is a do-while — see Question 28.

Part B · Additional Exercises

These questions cover the parts of Lectures 3 and 4 that Part A does not reach. Several ask you to record what the compiler or the runtime actually printed — copy those messages exactly, character for character, rather than paraphrasing them.

B1 · CONDITIONALS THAT COMPILE AND STILL DO THE WRONG THING

None of the programs in this group fails to compile, which is what makes this class of mistake expensive — nothing warns you. Predict what each one does before you run it; comparing your prediction with the result is where the value is.

Question 21 · Three Mistakes That Compile Perfectly

Each of the three fragments below compiles cleanly. Two of them then do something other than what they look like they do; the third does the right thing in a way you would not want to maintain. Predict the output of each one in writing, then run it and compare.

THE THREE FRAGMENTS

```
// (a)
double radius = -5;
if (radius >= 0);
{
    double area = radius * radius * Math.PI;
    System.out.println("Area is " + area);
}

// (b)
boolean even = false;
if (even = true) {
    System.out.println("reported even");
}

// (c)
int i = 5, count = 0;
if (i > 0) {
    System.out.println("A");
    count++;
} else {
    System.out.println("B");
    count++;
}
```

(a) The radius is negative and an area is printed anyway. Say exactly what the semicolon is, in language terms, and why the block below it runs.

(b) This prints its message and is not a compile error. Explain what `even = true` evaluates to and what the value of `even` is afterwards. Then say why `if (even == true)` is also worth avoiding, even though it works.

(c) Nothing here is wrong at run time — it is a maintenance fault. Rewrite it so `count++` appears once, and say in one sentence what that buys you.

Finally: which of the three would a compiler warning have caught? Turn on your IDE's inspections and check whether it flags any of them.

Question 22 · The Dangling Else

The fragment below is indented to suggest one reading and is compiled as another. With `x` set to 3 and `y` set to 2, predict what it prints before you run it.

DANGLINGELSE.JAVA

```
int x = 3, y = 2;

if (x > 2)
    if (y > 2) {
        int z = x + y;
        System.out.println("z is " + z);
    }
else
    System.out.println("x is <= 2");
```

(a) Run it. The output contradicts the indentation. State the rule that decides which `if` an `else` belongs to.

(b) Add braces so that the `else` belongs to the outer `if` instead. What does the program print now, and why is that surprising the first time you see it?

(c) Rewrite it once more so the `else` genuinely belongs to the inner `if` and reports that `x` is above 2 but `y` is not.

Then adopt the habit: always brace the body of an `if`, even when it is one statement. This question is the reason.

B2 · SWITCH THREE WAYS

The classic switch statement, the fall-through that catches everyone, and the switch expression the syllabus names for this week. Question 24 is the form you will meet in most Java written since about 2020.

Question 23 · Fall-Through, First by Accident and Then on Purpose

The switch below has no break statements at all. With a month of 2, predict its output before running it.

THE SWITCH WITH NO BREAKS

```
switch (month) {
    case 12: System.out.print("December ");
    case 1:  System.out.print("January ");
    case 2:  System.out.print("February ");
    default: System.out.print("(and on it falls)");
}
```

- (a) Run it with month 2, then with month 12, and explain in one sentence what a case label actually is.
- (b) Now use fall-through deliberately. Write a switch that reads a month number and prints the season, grouping the cases that share an outcome — case 12: case 1: case 2: and so on.
- (c) Extend it to print the number of days in that month, reading a year as well so February can be answered correctly. Group the 31-day months and the 30-day months; February gets its own case with the leap-year test from Question 2.

EXPECTED OUTPUT (EXAMPLE)

```
Enter a month number (1-12) and a year: 2 2026
(a) without break, month 2 gives: February (and on it falls)
(b) season: Winter
(c) days in month 2 of 2026: 28
```

Question 24 · From switch Statement to switch Expression

Java 14 added a form of switch that produces a value rather than performing statements. It cannot fall through, so no break is needed, and the compiler checks that every path yields something.

Rewrite your answer to Question 6 in this form.

THE SHAPE OF IT

```
String animal = switch (year % 12) {
    case 0 -> "Monkey";
    case 1 -> "Rooster";
    // ...
    default -> "Sheep";
}; // <-- a semicolon: the whole switch is an expression
```

Read three things at the start: a year for parts (a) and (c), a month number for parts (b) and (c), and a command word for part (e).

- (a) Write the zodiac version and confirm it gives the same answers as your Question 6 program for 2024, 2026 and 1900.
- (b) Use comma-separated labels to write the season switch from Question 23 as an expression: case 12, 1, 2 -> "Winter"; and so on.

- (c) Write the days-in-month switch as an expression. February needs a block branch, because it has a calculation to do first — and a block branch must end with `yield`, not `return`. Find out what the compiler says if you write `return` there instead.
- (d) Give a branch of your season switch `throw new IllegalArgumentException(...)` instead of a value, and say why the compiler accepts that in a position where a `String` was expected.
- (e) A switch selector may be a `String`. Write a three-case switch expression over the commands "start", "stop" and anything else. Then find the slide from Lecture 3 that lists every legal selector type and note how much longer that list is than the one most textbooks give.

EXPECTED OUTPUT (EXAMPLE)

```
Enter a year, a month number and a command: 2024 2 stop
(a) Dragon
(b) Winter
(c) days: 29
(e) stopping
```

B3 · LOGICAL OPERATORS

Question 25 · & Against &&, and Why It Matters

`&` and `&&` produce the same answer on the same operands. They differ in whether the right-hand operand is evaluated at all, and that difference is not academic.

THE EXPERIMENT

```
int x = 1;
boolean a = (x > 1) & (x++ < 10);
System.out.println("a = " + a + ", x = " + x);

int y = 1;
boolean b = (y > 1) && (y++ < 10);
System.out.println("b = " + b + ", y = " + y);
```

- (a) Both left operands are false, so `a` and `b` are both false. Run it and explain why `x` and `y` end up different.
- (b) Write a program with `int total = 50, count = 0;` that prints the average only when there is something to average, guarded by `count != 0 && total / count > 10`. Then swap the two operands and run it again. Record the exception you get, and say why `&` would fail in the same way.
- (c) With `age` at 24 and `weight` at 140, evaluate the four expressions below on paper, then run them. Two of the four caught people out in Lecture 3.

THE FOUR EXPRESSIONS

```
!(age > 18)
(age > 14) && (weight >= 150)
(age > 14) || (weight >= 150)
(age > 34) ^ (weight > 140)
```

- (d) Print every number from 1 to 12 that is divisible by 2 or by 3 but not by both, using `^` and no other logical operator.

EXPECTED OUTPUT — PART (A)

```
a = false, x = 2
b = false, y = 1
```

Question 26 · The Conditional Operator and Precedence

`condition ? valueIfTrue : valueIfFalse` is the only operator in Java that takes three operands. It is an expression, so it can go anywhere a value can go.

- (a) Read two integers and print the larger, twice: once with an `if-else` that assigns to a variable, and once with the conditional operator inside the `println` call.
- (b) Print whether the first number is odd or even, using the operator inside a string concatenation rather than a second statement.
- (c) Print `1 file copied` for a count of 1 and `3 files copied` for a count of 3, from one `printf` call with no `if` anywhere.
- (d) `+` binds more tightly than `?:`. Predict what `"Result: " + score > 40 ? "pass" : "fail"` does, then try to compile it and record the message. Fix it two ways — with parentheses, and by assigning to a variable first — and say which you would rather read six months from now.
- (e) `&&` binds more tightly than `||`. Given `weekday true`, `holiday false` and `urgent true`, say what `weekday && !holiday || urgent` evaluates to and where the implied parentheses go. Add them and confirm the answer does not change.

EXPECTED OUTPUT — PARTS (A) TO (C)

```
Enter two integers: 9 5
(a) max by if-else: 9
(a) max by ?:      : 9
(b) 9 is odd
(c) 1 file copied
```

B4 · LOOP MECHANICS

Three questions about the machinery itself rather than about what the loop computes. Two of them are places where a program compiles and then does something other than what it looks like it does; the third is about choosing the right loop in the first place.

Question 27 · One Semicolon, Three Different Outcomes

The same stray semicolon after a loop header means three different things in the three loop forms.

THE THREE CASES

```
// (a) with the semicolon, and then without it
int i;
for (i = 0; i < 3; i++);
{
    System.out.println("i = " + i);
}

for (int n = 0; n < 3; n++) {
    System.out.println("n = " + n);
}

// (b)
int k = 0;
while (k < 3);
{
    k++;
}

// (c)
int m = 0;
do {
    System.out.println("m = " + m);
    m++;
} while (m < 3);
```

(a) Run it. How many lines print, and what is the value of `i` when the block runs? Explain what the loop actually did.

(b) Do not run this one until you have predicted it, and be ready to stop it with Ctrl+C. Why does (b) behave differently from (a), when the semicolon is in the same place?

(c) Here the semicolon is not a mistake at all — it is required. Say what it terminates, and remove it to see what the compiler says.

EXPECTED OUTPUT — PARTS (A) AND (C)

```
(a) for with a stray semicolon:
    this block ran once, i = 3
(a) for without it:
    n = 0
    n = 1
    n = 2
(c) do-while, where the semicolon belongs:
    m = 0
    m = 1
    m = 2
```

Question 28 · do-while Where It Actually Belongs

Input validation is the case `do-while` was made for: you cannot test the answer until you have asked the question.

(a) Write a `do-while` loop that asks for a score between 0 and 100, reports any value outside that range, and keeps asking until it gets a usable one. There must be exactly one copy of the prompt in your source.

(b) Write the same validation with a `while` loop. Count how many times the prompt appears in your source now. That difference is the whole argument.

(c) Run a `do-while` whose condition is false from the start, and then a `while` with the same false condition, printing how many times each body ran. State the rule in one sentence.

(d) Now write `while (false) { passes++; }` with the literal, and compile it. Record the error. Then write `do { passes++; } while (false);` and note that it compiles. Explain the difference in terms of which statements the compiler can prove are reachable.

EXPECTED OUTPUT — PART (A)

```
Enter a score between 0 and 100: 150
150 is out of range.
Enter a score between 0 and 100: -4
-4 is out of range.
Enter a score between 0 and 100: 87
Accepted: 87
```

Question 29 · The continue Asymmetry

A for loop and the while loop you would write to replace it are usually interchangeable. `continue` is the one place where they are not.

THE TWO LOOPS

```
// (a) prints 1 2 4 5 7 8 10
for (int i = 1; i <= 10; i++) {
    if (i % 3 == 0) continue;
    System.out.print(i + " ");
}

// (b) the obvious translation
int i = 1;
while (i <= 10) {
    if (i % 3 == 0) continue;
    System.out.print(i + " ");
    i++;
}
```

(a) Run the `for` version and confirm the output.

(b) Predict what the `while` version does before running it. Add a pass counter and a guard so that it stops after fifteen passes, then run it and say precisely where it gets stuck and why.

(c) Repair the `while` version so it prints the same thing as the `for` version. There is more than one way; whichever you choose, the update has to happen somewhere `continue` cannot skip.

(d) Counting from 1 to 20 this time: use `break` to stop as soon as a running total passes 100 and report where it stopped. Then, over the same range, use `continue` to sum every value except the multiples of 10. Say in one sentence what each keyword does to the loop.

EXPECTED OUTPUT (EXAMPLE)

```
(a) for   : 1 2 4 5 7 8 10
(b) while: 1 2 <- stuck at i = 3 after 15 passes
(c) while: 1 2 4 5 7 8 10
(d) break  : stopped at n = 14, sum = 105
(d) continue: skipped 10 and 20, sum = 180
```

B5 · LOOPS AND FLOATING-POINT NUMBERS

Question 30 · Why a Loop Counter Must Not Be a double

The sum $0.01 + 0.02 + 0.03 + \dots + 1.00$ has an exact answer of 50.50, and there are exactly 100 terms. The loop below produces neither.

THE LOOP THAT LOOKS RIGHT

```
double sum = 0;
int passes = 0;
for (double d = 0.01; d <= 1.0; d += 0.01) {
    sum += d;
    passes++;
}
System.out.println("sum = " + sum + " after " + passes + " passes");
```

- (a) Run it. Report both the sum and the pass count. The pass count is the more informative of the two — explain what it tells you about where the error comes from.
- (b) Run the same series counting down, from 1.0 to 0.01. You get a different wrong answer. Is either of the two closer to correct, and does that tell you anything useful?
- (c) Rewrite it with an `int` counter from 1 to 100, computing the term inside the body as `i * 0.01`. Confirm it gives exactly 100 passes and 50.50.
- (d) Now the equality trap. Start `item` at 1.0 and subtract 0.1 in a `while (item != 0)` loop, printing `item` after 9, 10, 20 and 30 subtractions. Add a pass guard so you can stop it. Explain why the loop never ends, using the value you printed after 10 subtractions.
- (e) Replace the test with `Math.abs(item) > 1e-9` and confirm it terminates. State the rule for comparing two doubles in one sentence.

EXPECTED OUTPUT (ABRIDGED)

```
(a) counting up : sum = 49.50000000000003 after 99 passes
(b) counting down: sum = 50.48999999999995 after 99 passes
(c) int counter : sum = 50.50 after 100 passes
(d) after 9 subtractions item = 0.1000000000000014
(d) after 10 subtractions item = 1.3877787807814457E-16
(e) with a tolerance the loop ended after 10 subtractions
```